

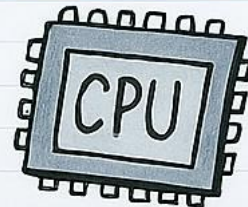
DEFINITION



Data Processing:

The transformation or manipulation of input data into meaningful output using arithmetic and logical operations.

CORE HARDWARE COMPONENTS



• Arithmetic Logic Unit (ALU)

Executes arithmetic (addition, subtraction), and logical (AND, OR) operations.

• Registers

Small, high-speed storage units that temporarily hold operands and results of ALU operations.

• Control Unit (CU)

Directs data flow between registers and ALU during instruction execution.

• System Clock

Synchronizes operations of all processing components

ASSIGNMENT: THE FOUR FUNDAMENTAL FUNCTIONS OF A COMPUTER

*A Computer Organization
and Security Language
Perspective*

INSTRUCTION SET ARCHITECTURE (ISA) ROLE

The ISA defines **How** the CPU performs data processing through machine instructions.

Categories of processing instructions:

1. Arithmetic Instructions – ADD, SUB, MUL, DIV
2. Logical Instructions – AND, OR, XOR, NOT
3. Bit Manipulation – SHL, SHR, ROL, ROR
4. Comparison instructions – CMP, TEST

HARDWARE COMPONENTS

System Bus:
Transfers data, addresses, and control signals.

I/O Modules:
Interface between CPU and external devices.

Ports:
Logical connection points to external hardware.

INSTRUCTION SET ARCHITECTURE (ISA) ROLE

XX6 ASSEMBLY:
EXAMPLE:
I/O PORT COMMUNICATION

IN—Input data from an port (keyboard buffer)

OUT—Output data from accumulator to port.

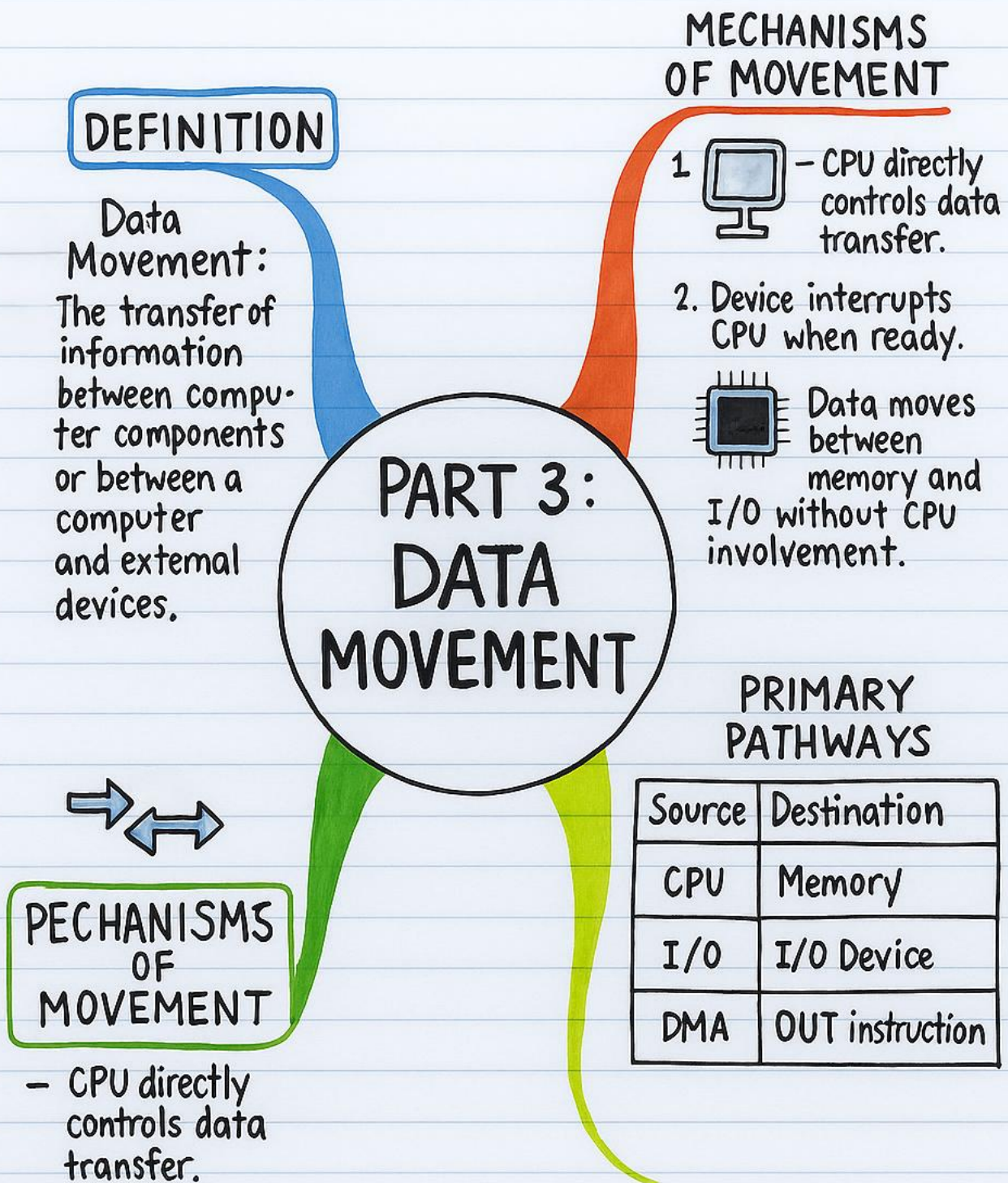
HARDWARE COMPONENTS & ISA ROLE

EXPLANATION

- in al, ax reads from an I/O port (keyboard buffer).
- out 0x64, al writes data to another port
- Demonstrates CPU-I/O interaction via port-mapped I/O using data bus.

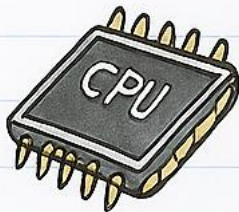


```
> section .text
> global _start
_start:
    mov dx, 0x60
    ; Keyboard data port (example address)
    in al, dx
    out 0x64, al
    ; Write same data to output port 0x64 (for demonstration)
; / Exit
    mov eax, 60
    xor edi, edi
    syscall
```



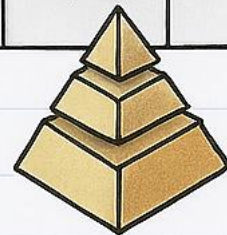
DEFINITION

Data Storage:
The retention of digital information for immediate or future use by the CPU or other devices.



STORAGE HIERARCHY

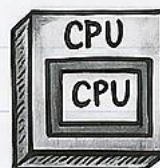
Level	Example	Speed
Registers	CPU registers	Fastest
Cache	L1/L2/L3	Very Fast
Main Memory (RAM)	DRAM	Moderate
Secondary Storage	SSD/HDD	Slow



PART 2: DATA STORAGE

HARDWARE COMPONENTS

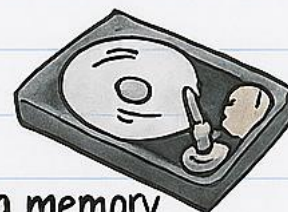
- **REGISTERS:**
Temporary fastest CPU storage.
- **CACHE MEMORY:**
Intermediate storage to reduce memory access latency.
- **RAM:**
Main working memory for active processes.

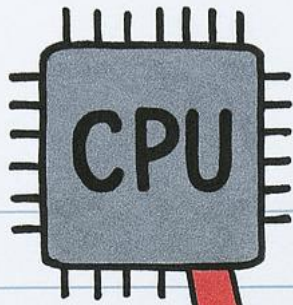


- **RAM:**
Main working memory for active processes.

STORAGE DRIVES

- Long-term data preservation.



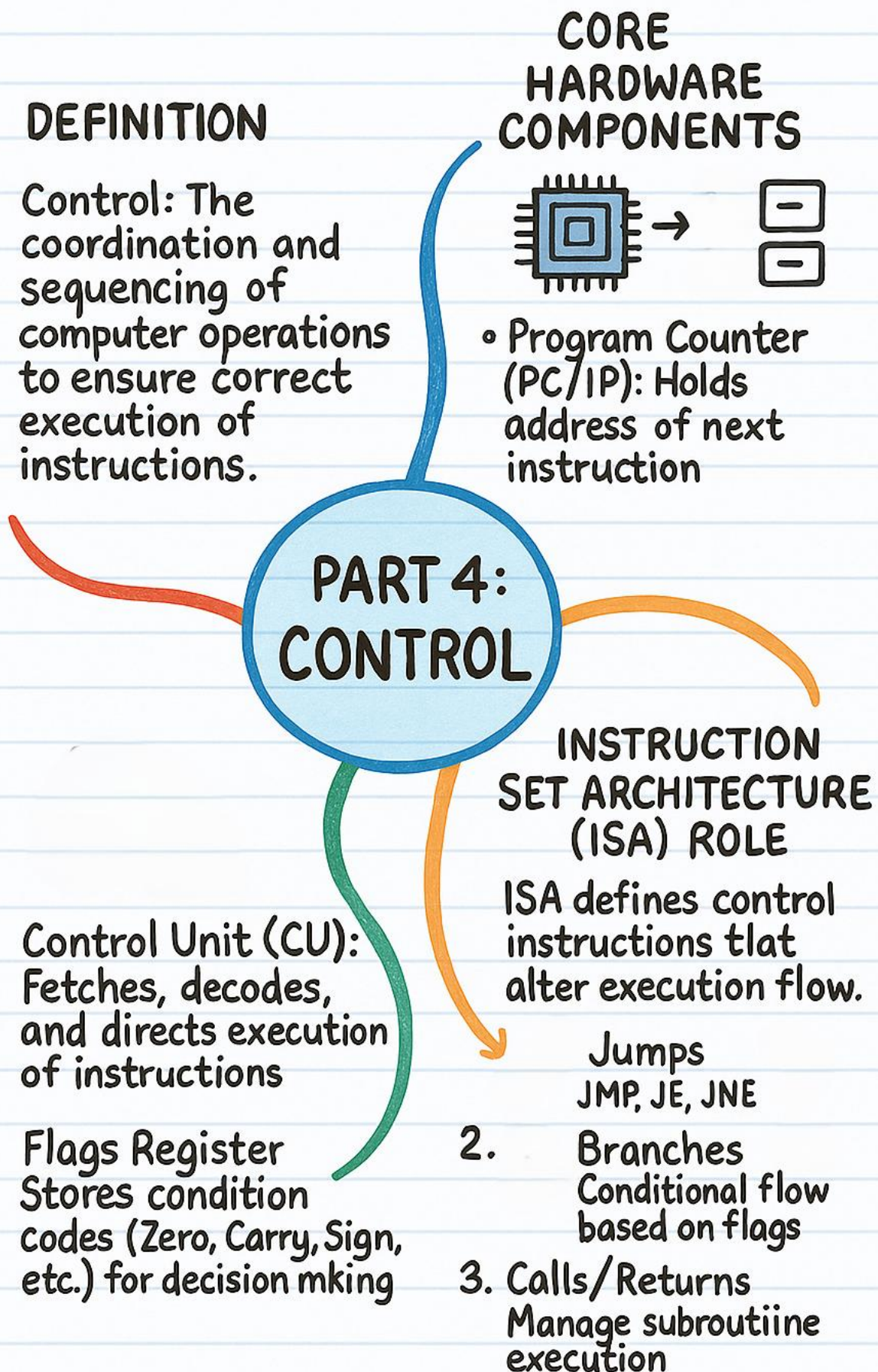


x86 Assembly Example: Arithmetic Operation

```
section .data
    num1 db 10
    num2 db 20
    result db 0
section .text
    global _start
    mov eax, 60
    syscall: exit
    xor edi, edi
    syscall
```

Explanation:

- **mov** – Transfers data between registers and memory.
- **add** – ALU performs addition of operands.
- The ALU executes the operation; registers hold operands; the flow.



x86 Assembly Example: Conditional Jump

```

section .data
a db 5
b db 5
equal_msg db "Values are equal"
notequal_msg db "Values are not equal"

section .text
global _start
_start:  flags (Load first value
        mov al, (b) (Load second value
        cmp al, bl (Compare AL and
        je equal_label (Jump if equal)
        jmp notequal_label : Otherwise jump
                        to not equal
equal_label:
    /Code executed when equal
    mov eax, 60
    xor edi, edi
    syscall
  
```

Control Flow & Summary

Explanation

- ('cmp' sets flags in the CPU (Zero Flag, Sign Flag))
- 'je' checks Zero Flag (ZF=1 means equal)
- The Instruction Pointer (IP) changes to target label when condition is true
- Demonstrates control flow manipulation by Control Unit

101

Summary

Function	Core Hardware	ISA Role
Data, Processing	ALU, Registers	Arithmetic/Logic Ops
Data Storage	Registers, Cache, RAM	Load/Store Data
Data Movement	Buses, I/O Modules	IN, OUT
Control	CU, FG/ IP, IR	Execution Flow Control
		JMP, JE